

Pipeline de Processamento de PDFs

1. Visão Geral da Atividade

Iremos construir um pipeline full stack onde é possível fazer upload de documentos PDF (como faturas ou contratos). O sistema irá ler o conteúdo PDF, extrair metadados específicos (como o "Valor Total") utilizando Expressões Regulares (Regex) e armazenar essas informações numa base de dados MySQL para exibição.

Stack Tecnológica

- Ambiente de Desenvolvimento: VS Code
- Back-end: Node.js com Express
- Processamento e arquivos: Multer (Upload) e pdf-parse (Leitura do PDF)
- Base de Dados: MySQL (Módulo mysql2 com Pool de conexões)
- Front-end: HTML, CSS e JavaScript Vanilla (Fetch API)
- Produtividade: Nodemon (Live reload do servidor)

2. Preparação da Base de Dados (MySQL)

Abra um cliente MySQL como o Workbench e execute o seguinte script para criar a base de dados e a tabela necessária para a solução proposta:

```
CREATE DATABASE IF NOT EXISTS pdf_pipeline;
USE pdf_pipeline;

CREATE TABLE IF NOT EXISTS documentos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome_arquivo VARCHAR(255) NOT NULL,
  valor_extraido DECIMAL(10,2),
  data_upload TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. Inicialização e Estrutura do Projeto no VS Code

No terminal do VS Code, crie a pasta do projeto, inicialize o ambiente Node.js e instale as dependências (incluindo o nodemon globalmente para facilitar o reinício do servidor):

```
mkdir processador-pdf
cd processador-pdf
npm init -y
npm install express multer pdf-parse mysql2 cors
npm install -g nodemon
```

```
mkdir public
```

A estrutura final das pastas no VS Code ficará assim:

```
processador-pdf/  
├── node_modules/  
├── public/  
│   ├── index.html  
│   └── app.js  
├── package.json  
└── server.js      (nosso servidor Node.js principal)
```

4. Desenvolvimento do Servidor (server.js)

Este é o núcleo da implementação. O arquivo `server.js` conectará todas as pontes: rotas, upload, leitura do PDF e base de dados. Este código implementa boas práticas de produção, como o uso de Pool de conexões, processamento assíncrono do File System e um bloco "finally" para garantir que nenhum arquivo temporário fique esquecido no disco, mesmo em caso de erro.

```
const express = require('express');  
const multer = require('multer');  
const pdfParse = require('pdf-parse');  
const mysql = require('mysql2/promise');  
const cors = require('cors');  
const fs = require('fs');  
const path = require('path');  
  
const app = express();  
app.use(cors());  
app.use(express.json());  
  
// Servir a pasta public como arquivos estáticos para o front-end  
app.use(express.static(path.join(__dirname, 'public')));  
  
// 1. GARANTIR QUE A PASTA UPLOADS EXISTE (Evita erro ENOENT)  
const dirUploads = path.join(__dirname, 'uploads');  
if (!fs.existsSync(dirUploads)){  
    fs.mkdirSync(dirUploads);  
}  
  
// 2. CONFIGURAÇÃO DA BASE DE DADOS MYSQL COM POOL  
// O Pool é mais eficiente e evita o esgotamento de conexões  
const pool = mysql.createPool({  
    host: 'localhost',
```

```

    user: 'root',
    password: '', // Inserir a chave de acesso ao MySQL
    database: 'pdf_pipeline',
    waitForConnections: true,
    connectionLimit: 10,
    queueLimit: 0
  });

// 3. CONFIGURAÇÃO DO MULTER PARA UPLOAD DE ARQUIVOS
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, 'uploads/');
  },
  filename: (req, file, cb) => {
    cb(null, Date.now() + '-' + file.originalname);
  }
});
const upload = multer({ storage: storage });

// 4. ROTA DE UPLOAD E PROCESSAMENTO DE PDF
app.post('/api/upload', upload.single('pdfDocument'), async (req, res) => {
  let filePath = null; // Declarado fora do try para o finally ter acesso

  try {
    if (!req.file) {
      return res.status(400).json({ error: 'Nenhum arquivo enviado.' });
    }

    filePath = req.file.path;
    const fileName = req.file.originalname;

    // Ler o arquivo PDF fisicamente do disco de forma assíncrona
    const dataBuffer = await fs.promises.readFile(filePath);
    const pdfData = await pdfParse(dataBuffer);
    const extractedText = pdfData.text;

    // Expressões Regulares (Regex) avançada e Robusta (Usada para cobrir
    todos os casos e obter o valor):
    const regexValor = /R$\s*([0-9]{1,3}(\?:\.[0-9]{3})*,[0-9]{2}|[0-9]+
    (\?:\.[0-9]{2})?)/i;
    const match = extractedText.match(regexValor);

    let valorFinal = null;
    if (match && match[1]) {
      // Limpeza e formatação do número para o MySQL (trocar vírgula por
      ponto)
      valorFinal = parseFloat(match[1].replace(/\./g, '').replace(',',
      '.'));
    }
  }

```

```

    // Inserção na Base de Dados usando o Pool
    const [result] = await pool.execute(
        'INSERT INTO documentos (nome_arquivo, valor_extraido) VALUES
(?, ?)',
        [fileName, valorFinal]
    );

    res.json({
        message: 'arquivo processado com sucesso!',
        nomeArquivo: fileName,
        valorExtraido: valorFinal
    });

} catch (error) {
    console.error("Erro durante o fluxo:", error);
    res.status(500).json({ error: 'Erro ao processar o arquivo PDF.' });
} finally {
    // 5. LIMPEZA ASSÍNCRONA E SEGURA DO DISCO
    // O bloco finally SEMPRE executa, mesmo se ocorrer um erro.
    if (filePath) {
        try {
            // Remove o arquivo de forma assíncrona para não bloquear o
Node.js
            await fs.promises.unlink(filePath);
        } catch (unlinkError) {
            console.error("Alerta Crítico: Falha ao limpar o disco:",
unlinkError);
        }
    }
}
});

// 6. ROTA PARA LISTAR OS DOCUMENTOS PROCESSADOS
app.get('/api/documentos', async (req, res) => {
    try {
        const [rows] = await pool.execute('SELECT * FROM documentos ORDER BY id
DESC');
        res.json(rows);
    } catch (error) {
        res.status(500).json({ error: 'Erro ao procurar documentos na base de
dados.' });
    }
});

// 7. INICIAR O SERVIDOR
const PORT = 3000;
app.listen(PORT, () => {
    console.log(`Servidor a correr em http://localhost:${PORT}`);
});

```

5. Desenvolvimento do Front-end

Dentro da pasta "public", criaremos o HTML para o formulário e o Javascript para interagir com o nosso servidor Node via Fetch API.

5.1 O arquivo public/index.html

```
<!DOCTYPE html>
<html lang="pt-PT">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Pipeline de Faturas e Contratos</title>
  <style>
    body { font-family: Arial, sans-serif; margin: 40px; background-color:
    #f4f7f6; }
    .container { max-width: 800px; margin: auto; background: white; padding:
    20px; border-radius: 8px; box-shadow: 0 4px 8px rgba(0,0,0,0.1); }
    h1, h2 { color: #333; }
    .upload-section { border: 2px dashed #ccc; padding: 20px; text-align:
    center; border-radius: 6px; margin-bottom: 20px; }
    button { background-color: #0055A4; color: white; border: none; padding:
    10px 15px; border-radius: 4px; cursor: pointer; }
    button:hover { background-color: #003366; }
    table { width: 100%; border-collapse: collapse; margin-top: 20px; }
    th, td { padding: 12px; border: 1px solid #ddd; text-align: left; }
    th { background-color: #f8f9fa; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Dashboard de Faturas em PDF</h1>

    <div class="upload-section">
      <input type="file" id="pdfInput" accept="application/pdf">
      <button onclick="fazerUpload()">Processar PDF</button>
      <p id="statusMensagem" style="color: #666;"></p>
    </div>

    <h2>Documentos Processados</h2>
    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Nome do arquivo</th>
          <th>Valor Extraído (R$)</th>
          <th>Data de Upload</th>
        </tr>
```

```

        </thead>
        <tbody id="tabelaDocumentos">
            <!-- Linhas serão inseridas aqui via JavaScript -->
        </tbody>
    </table>
</div>

    <script src="app.js"></script>
</body>
</html>

```

5.2 O arquivo public/app.js

Este arquivo faz a ligação assíncrona. Ele envia o PDF (usando FormData para criar o multipart/form-data exigido pelo Multer) e atualiza a tabela na interface.

```

// Função para enviar o arquivo ao back-end
async function fazerUpload() {
    const fileInput = document.getElementById('pdfInput');
    const statusText = document.getElementById('statusMensagem');

    if (fileInput.files.length === 0) {
        statusText.innerText = "Por favor, selecione um arquivo PDF primeiro.";
        return;
    }

    // Usando FormData para enviar via multipart/form-data
    const formData = new FormData();
    formData.append('pdfDocument', fileInput.files[0]);

    statusText.innerText = "Realizando upload e processamento... (Aguarde)";

    try {
        const response = await fetch('/api/upload', {
            method: 'POST',
            body: formData
        });

        const data = await response.json();

        if (response.ok) {
            statusText.innerText = `Sucesso! Extraído: R$ ${data.valorExtraido}
|| 'Não encontrado'`;
            carregarDocumentos(); // Atualiza a tabela imediatamente
            fileInput.value = ""; // Limpa o input
        } else {
            statusText.innerText = `Erro: ${data.error}`;
        }
    } catch (error) {

```

```

        console.error("Erro na requisição:", error);
        statusText.innerText = "Erro ao conectar com o servidor.";
    }
}

// Função para buscar os dados do MySQL (via Back-end) e preencher a tabela HTML
async function carregarDocumentos() {
    try {
        const response = await fetch('/api/documentos');
        const documentos = await response.json();

        const tbody = document.getElementById('tabelaDocumentos');
        tbody.innerHTML = ''; // Limpa a tabela

        documentos.forEach(doc => {
            const tr = document.createElement('tr');

            // Formatar a data
            const dataFormatada = new Date(doc.data_upload).toLocaleString('pt-PT');

            // Lidar com valores nulos caso a Regex não encontre nada
            const valorFormatado = doc.valor_extraido !== null ?
doc.valor_extraido : 'Não identificado';

            tr.innerHTML = `
                <td>${doc.id}</td>
                <td>${doc.nome_arquivo}</td>
                <td>${valorFormatado}</td>
                <td>${dataFormatada}</td>
            `;
            tbody.appendChild(tr);
        });
    } catch (error) {
        console.error("Erro ao carregar documentos:", error);
    }
}

// Carregar a lista inicial assim que a página abrir
window.onload = carregarDocumentos;

```

6. Testando a Implementação

Para provar que o pipeline funciona de ponta a ponta, siga o seguinte roteiro de teste:

1. Inicie o servidor abrindo o terminal e rodando:

`nodemon server.js`

2. Abra o navegador e utilize: `http://localhost:3000`
3. Crie um documento falso contendo a frase: "O Valor Total da fatura é de R\$ 3.450,90 a ser pago nesta data." e guarde como PDF.
4. Faça o upload pelo Front-end e observe o valor de R\$ 3450.90 aparecer na tabela de documentos persistidos no MySQL.
5. Verifique que a pasta "uploads" permanece limpa após o arquivo ser lido com sucesso (ou mesmo que ocorra um erro de extração), demonstrando a arquitetura à prova de falhas do bloco finally.